Autonomous First Responder Fire & Rescue Robot

Combined Module Documentation Report

Group 7
Gebze Technical University
Computer Engineering Department
CSE 396 Computer Engineering Project

April 2026

Contents

1 MOD-01: Embedded & Hardware Module

1.1 Authors and Responsibilities

Name	Student ID	Role
Nuri Ziya Kırtepe	210104004027	Primary - Power & UART
Ömer Nacar	210104004814	Secondary - Motors & Stuck Detection
Gabil Rahimli	230104004902	Secondary - Env. Sensors

Table 1: MOD-01 Team Responsibilities

1.2 Module Overview

The Embedded and Hardware Control Module (MOD-01) serves as the "reflexive core" of the search and rescue robot. It is responsible for low-level hardware abstraction, real-time motor control, environmental sensing, and safety failsafes. Built on the STM32F103C8T6 (Blue Pill) microcontroller, this module ensures that high-level commands from the Raspberry Pi 5 are translated into physical motion while simultaneously monitoring the robot's health and surroundings through a structured telemetry stream.

1.3 Key Responsibilities

The functional scope of MOD-01 includes the following technical domains:

- **4WD Motor Control:** Driving four DC motors via the L298N H-Bridge using high-frequency PWM.
- **Stuck Detection:** Real-time IMU-based monitoring to detect and recover from mechanical obstructions.
- **Environmental Sensing:** Polling DHT22 (Temp/Humidity) and MQ-2 (Smoke/-Gas) sensors.
- **Power Management:** Managing a dual-powerbank setup with a 12V decoy trigger and time-based operation limits (60-minute RTH limit).
- **Telemetry Hub:** Aggregating sensor data into a 50Hz structured UART stream for the Raspberry Pi 5.

1.4 Internal Responsibilities and Interface Design

1.4.1 Power Management (pwr_management.h)

This sub-module handles the 12V Type-C Decoy trigger via `GPIOA_PIN_5`. It monitors the operational duration to enforce a strict safety limit, ensuring the robot initiates Return-to-Home (RTH) protocols before battery depletion.

1.4.2 UART Communication (uart_comm.h)

A 115200 baud UART bridge is maintained for bidirectional communication. MOD-01 serializes environmental data, IMU yaw angles, and ultrasonic distances into a packet structure transmitted at a 20ms period.

1.4.3 Motor and Stuck Logic (motor_control.h & stuck_detection.h)

Chassis movement is governed by mapping UART directions (Forward, Backward, Left, Right) to L298N GPIO states. Parallel to this, the MPU6050 IMU is polled via I2C to verify motion. If the motors are active but the accelerometer/gyroscope magnitude falls below defined thresholds (50mg / 5 deg/s) for over 2 seconds, a recovery sequence is triggered.

1.4.4 Environmental Drivers (environment_sensors.h)

High-level drivers facilitate data acquisition from the DHT22 and MQ-2 sensors. The MQ-2 analog signal is processed through the STM32 ADC, utilizing an internal threshold logic to flag smoke alerts.

1.5 Hardware Architecture and Physical Implementation

The physical layer follows a strict architectural scheme designed for isolation and safety:

- **Dual Power Isolation:** To prevent brownouts caused by motor stall currents, a dedicated 10000mAh PD powerbank is used for the 4WD system, while a separate supply powers the Pi 5 and STM32.
- **Common Ground Policy:** All power and logic grounds are interconnected to establish a stable reference for UART and I2C communications.
- **Voltage Protection:** 1k/2k Ohm voltage dividers are implemented for non-5V-tolerant pins (e.g., MQ-2 analog out and Ultrasonic Echo) to protect the 3.3V STM32 logic.
- **Pull-up Configuration:** A 4.7k Ohm resistor is utilized between VCC and the Data line of the DHT sensor to ensure signal integrity.

1.6 Software Interface Summary (API)

The following table summarizes the primary public functions exposed by the module:

Function	Description	Header
`pwr_init()`	Configures power GPIOs and failsafe timers.	`pwr_management.h`
`uart_send_telemetry()`	Packages and transmits sensor data via DMA.	`uart_comm.h`
`motor_set_state()`	Sets speed and direction for the 4WD chassis.	`motor_control.h`
`stuck_check()`	Validates motion using IMU feedback.	`stuck_detection.h`
`env_read_all()`	Reads all environmental telemetry in one call.	`environment_sensors.h`

Table 2: MOD-01 API Summary

1.7 Input / Output Summary

Type	Source/Target	Description
Input	Sensors	ADC (MQ-2), I2C (IMU), GPIO (Ultrasonic/DHT).
Input	MOD-04 (Pi 5)	Direction and Speed commands via UART.
Output	L298N Driver	PWM duty cycles and H-Bridge logic pins.
Output	MOD-04 (Pi 5)	Aggregated telemetry JSON/Struct over UART.
Output	Indicators	LED and Buzzer status feedback.

Table 3: MOD-01 Input/Output Interfaces

1.8 Known Risks and Mitigations

- **Motor Drift:** Mechanical differences in motors cause the robot to veer. *Mitigation:* Implementation of `motor_set_trim()` to balance PWM signals.
- **IMU Gyro Drift:** Yaw angle accumulates error over time. *Mitigation:* Future integration of accelerometer-based tilt compensation.
- **UART Packet Loss:** Electrical noise may corrupt data. *Mitigation:* Use of twisted-pair wiring and future software-side checksums.

1.9 Conclusion

MOD-01 provides the essential physical foundation for the search and rescue robot, ensuring that high-level autonomous intent is executed reliably. By centralizing real-time motor control, environmental sensing, and hardware-level failsafes into the STM32 firmware, the system maintains a robust response to both internal faults and external hazards, laying the groundwork for successful mission deployment.

2 MOD-02: AI & Vision Pipeline

2.1 Authors and Responsibilities

Name	Student ID	Role
Fatma Öztürk	230104004152	Primary - AI Pipeline & Model Selection
Gabil Rahimli	230104004902	Primary - AI Pipeline, Model Selection & YOLO Training
Evrim Doğa Solmaz	230104004042	Primary - Vision Integration & Hardware Interfacing
Tuana Melisa Aksoi	230104004903	Secondary - Model Testing & Severity Classification
Uğur Anıl Güney	210104004011	Secondary - Dataset Preparation & YOLO Training
Dicle Çoban	220104004088	Secondary - Performance Optimization & Resource Management

Table 4: MOD-02 Authors and Responsibilities

2.2 Module Description

Module 2 is responsible for the visual perception and victim-analysis layer of the robot. It runs on the Raspberry Pi 5 and processes live RGB frames captured by the Pi Camera Module V3. The main purpose of this module is to detect human presence in real time and classify the detected person as TRAPPED, LYING, STANDING, or NONE. Since the project targets disaster environments where cloud connectivity may be unavailable, the entire pipeline is designed to work on-device as an edge-AI component.

Within the overall system, Module 2 acts as the robot's visual intelligence layer. It transforms raw image frames into structured victim information that can be used by the higher-level FSM, the Web Dashboard, and the Unity Digital Twin.

2.3 Main Responsibilities

The main responsibilities of Module 2 are:

- Initializing and managing the Pi Camera Module V3,
- Capturing live frames from the environment,
- Detecting human candidates using a YOLO-based detector,
- Selecting the most relevant target when multiple people appear,
- Classifying victim condition using an edge-side analysis backend,
- Producing structured output for upper modules,

- Supporting dynamic FPS switching for resource-aware execution,
- Pausing and resuming the vision pipeline during STT execution.

2.4 Hardware and Software Stack

This module is deployed on a **Raspberry Pi 5 (8GB)** with an Active Cooler and uses the **Pi Camera Module V3** as its main visual input source.

The software stack is Python-based and includes:

- Python 3.10+
- OpenCV
- Ultralytics YOLOv8
- Moondream2 or an alternative lightweight edge-side classifier backend

This design supports the project goal of running perception directly on the robot under edge hardware constraints.

2.5 Public Interface and Data Contract

The public interface of the module is exposed through the vision pipeline abstraction. External modules are expected to communicate only with this public layer instead of directly calling internal helper files.

Method	Purpose
`initialize_camera()`	Initializes the camera and loads the required inference backends.
`get_latest_target()`	Returns the latest victim result, or None if no person is detected.
`pause_vision_pipeline()`	Temporarily stops frame processing and inference during STT execution.
`resume_vision_pipeline()`	Restarts the vision pipeline after the interruption ends.

Table 5: MOD-02 Public Methods

The main public output type is `TargetData`. It contains:

- `pos_x, pos_y`: pixel coordinates of the target,
- `distance_cm`: estimated distance to the target,
- `severity`: victim state,
- `confidence`: model confidence score.

This output is intentionally compact so it can be consumed easily by upper layers. It is also designed to remain compatible with the victim-status and visualization needs of the higher-level modules. The public API is intentionally kept minimal, while lower-level camera, detection, and analysis operations are handled internally by specialized helper modules.

2.6 Internal Architecture

The public orchestration point of the module is `ai_vision.py`. Internally, the module is divided into smaller files to separate responsibilities and improve maintainability:

- `camera_internal.py`: camera initialization, frame capture, FPS update, and cleanup.
- `human_detector_internal.py`: YOLO-based person detection and best-target selection.
- `victim_analyzer_internal.py`: victim-state classification and priority-compatible output generation.

In addition, `ai_vision.py` also contains runtime control logic such as exploration/assessment switching, dynamic FPS control, and pause/resume handling during STT interrupts.

2.7 Operational Flow

At runtime, the module first initializes the camera and required AI backends. It then continuously captures RGB frames from the environment. Each frame is processed by the human detector. If no person is found, the cycle ends without producing a target. If one or more people are detected, the most relevant candidate is selected and forwarded to the victim-analysis stage. The analysis result is then packaged into a `TargetData` object and delivered to upper modules and the FSM.

This makes Module 2 a perception pipeline rather than a simple camera reader. Its output directly supports target prioritization, visualization, and higher-level mission decisions.

2.8 Inter-Module Connections

Module 2 has several important connections within the project architecture:

- **Module 1 – Embedded & Hardware:** Module 2 relies on the hardware platform prepared by Module 1, including stable Raspberry Pi deployment, camera integration, power context, and robot-level embedded infrastructure.
- **Module 3 – Acoustics & Navigation:** Module 2 works together with Module 3 as part of the tri-modal sensor-fusion strategy. Visual detections trigger target prioritization, while Module 3 contributes acoustic localization.
- **Module 4 – Web Dashboard & STT:** This is the strongest software-level interaction. During STT execution, Module 2 may pause the vision pipeline to release CPU and RAM resources, then resume safely afterward.
- **Module 5 – Unity Digital Twin:** Module 5 consumes victim-status outputs from Module 2 to display color-coded target information. The severity output of Module 2 must remain compatible with Unity mapping.
- **Higher-Level FSM / Control Logic:** The FSM uses the structured outputs of Module 2 for victim scene analysis, evaluation, and action planning.

2.9 Input / Output Summary

Inputs: live RGB frames from the camera, runtime control commands such as pause/resume, and configuration parameters such as FPS or confidence thresholds.
Outputs: `TargetData` objects, victim severity labels, confidence values, estimated target position and distance, and integration-ready outputs for the dashboard, Unity, and FSM.

2.10 Testing and Validation

The following test scenarios are suitable for Module 2:

- **AV-01:** Pi Camera initializes successfully on Raspberry Pi 5.
- **AV-02:** Frames are captured continuously without freeze or crash.
- **AV-03:** YOLO detector identifies a human in a test frame.
- **AV-04:** The pipeline returns None when no human is present.
- **AV-05:** Victim analysis distinguishes TRAPPED, LYING, and STANDING.
- **AV-06:** Target selection behaves consistently in multi-person scenes.
- **AV-07:** Dynamic FPS switching works correctly.
- **AV-08:** Pause/resume works safely during STT execution.
- **AV-09:** Output format remains compatible with upper modules.

2.11 Risks, Limitations, and Conclusion

One important risk is thermal and memory pressure on the Raspberry Pi 5, especially during heavy victim-analysis inference. Another risk is latency if the selected model is too large for real-time execution. To reduce these problems, the module uses dynamic FPS management, active cooling, and pause/resume behavior during STT.

Overall, Module 2 is a central perception component of the project. It converts raw visual data into structured victim information and provides a reliable bridge between sensing and decision-making in the overall rescue workflow.

3 MOD-03: Acoustics & Navigation Module

3.1 Authors and Responsibilities

Name	Student ID	Role
Uğur Anıl Güney	210104004011	Primary - STM32 Firmware & Acoustic Processing
Evrim Doğa Solmaz	230104004042	Secondary - Python Bridge & Navigation Interfacing
Tuana Melisa Aksoy	230104004903	Secondary - FSM Branching & Mode Transitions
Dicle Çoban	220104004088	Secondary - Unity Visualizer & Beam Mapping

Table 6: MOD-03 Team Responsibilities

3.2 Module Overview

MOD-03 (Acoustics & Navigation) is the acoustic sensing and autonomous movement layer of the rescue robot. Its responsibility is to detect human distress calls using a three-microphone array, filter out continuous motor noise from the raw audio signal, compute the direction of the sound source, and translate that direction into motor navigation commands that steer the robot toward the victim.

MOD-03 spans three platforms and three programming languages: an STM32 Blue Pill microcontroller running C firmware for signal processing (`acoustics_iir.h`), a shared C header defining the FSM branching logic used by both the STM32 side and the Pi-side main FSM (`fsm_acoustic.h`), a Raspberry Pi 5 running a Python navigation bridge (`acoustic_homing.py`), and a Unity C# component for operator-facing visualization (`MapManager_AcousticBeam.cs`).

3.3 Purpose of the Module

The purpose of MOD-03 is to provide the robot with directional hearing capability and to connect acoustic detections to autonomous navigation decisions. The system's vision pipeline (MOD-02) requires a direct line of sight to detect a victim. Acoustic localization provides a complementary detection modality that does not depend on visibility, which is important in smoke-filled or obstructed disaster environments.

The core engineering challenge is that the robot's own DC motors and gearboxes generate continuous mechanical noise that masks incoming distress calls on the microphone array. MOD-03 solves this by running a Software-based Digital IIR Filter directly on the STM32, which attenuates motor and gearbox noise from the ADC samples before bearing computation is attempted. This is described in the project proposal as a deliberate choice over static hardware (RC) filters: unlike hardware filters, a software IIR filter can be tuned without physical changes to the circuit.

In addition to acoustic sensing, MOD-03 is also responsible for the initial area mapping behavior. At startup, before exploration begins, the robot executes a 360-degree Spin-Scan that populates a 2D occupancy grid using the four ultrasonic sensors. This grid is the spatial foundation on which the exploration FSM operates.

3.4 Internal Responsibilities of MOD-03

3.4.1 STM32 Acoustic Signal Processing (acoustics_iir.h)

This subsystem runs on the STM32 Blue Pill microcontroller. It owns the lowest-level acoustic signal pipeline, from raw ADC samples to a computed bearing angle and occupancy grid. The subsystem exposes four public functions:

- `acoustics_iir_filter_apply()`: Applies a 4th-order Software-based Digital IIR filter to the raw ADC sample buffer from each microphone. Its purpose is to attenuate motor and gearbox noise.
- `acoustics_compute_bearing()`: Computes the bearing angle to the acoustic source via phase difference. The result is stored in an `acoustics_result_t` struct. A detection is flagged when confidence meets or exceeds `ACOUSTICS_HIT_THRESHOLD = 0.6f`.
- `acoustics_spinscan_execute()`: Drives the robot through a 360-degree rotation while pinging ultrasonic sensors to populate an `acoustics_grid_t` occupancy grid.
- `acoustics_homing_navigate()`: Converts a confirmed bearing angle into a low-level motor direction command (`acoustics_nav_cmd_t`).

3.4.2 FSM Acoustic Branching (fsm_acoustic.h)

This subsystem defines the finite state machine branching logic. The header defines 11 FSM states shared across the project. Important constants:

- `FSM_ACOUSTIC_MIN_CONFIRMS = 3`: Must receive 3 consecutive A_Hit = 1 readings to guard against false positives.
- `FSM_ACOUSTIC_HOMING_TIMEOUT_MS = 30000`: Homing timeout fallback to EXPLORE.
- `FSM_ACOUSTIC_COOLDOWN_MS = 3000`: A 3-second cooldown between homing transitions.
- `FSM_BEARING_DEAD_ZONE_DEG = 10.0f`: Bearings within $\pm 10^\circ$ drive forward.

Public functions include `FSM_Acoustic_Init()`, `FSM_Acoustic_Update()`, `FSM_Acoustic_IsHomingTime` `FSM_Acoustic_ShouldInterruptExplore()`, `FSM_Acoustic_ResetStreak()`, and `FSM_Acoustic_GetSt`

3.4.3 Python Navigation Bridge (acoustic_homing.py)

Runs on the Raspberry Pi 5. Implements the `AcousticHomingBridge` class with methods:

- `process_telemetry()`: Called on every FSM tick with `AcousticTelemetry`. Manages the streak counter and enters homing mode.
- `notify_fsm_transition()`: Notifies the MOD-04 main FSM.
- `reset()`: Clears the hit streak counter and homing flag.

3.4.4 Unity Acoustic Beam Visualization (MapManager_AcousticBeam.cs)

Runs inside Unity. It exposes methods to visualize the bearing:

- `ShowAcousticBeam()`: Renders the bearing indicator.
- `HideAcousticBeam()`: Deactivates the beam.
- `UpdateAcousticBeamAngle()`: Updates the bearing.

3.5 Data Structures Used in MOD-03

3.5.1 C / STM32 Layer

- `acoustics_status_t`: Return code enum.
- `acoustics_result_t`: Stores bearing_deg, hit_detected, and timestamp_ms.
- `acoustics_grid_t`: 2D occupancy grid.
- `acoustics_nav_cmd_t`: Motor direction output enum.
- `fsm_state_t`: Defines all 11 top-level FSM states.
- `fsm_acoustic_event_t` & `fsm_acoustic_result_t`: FSM tracking payloads.

3.5.2 Python / Raspberry Pi Layer

- `AcousticTelemetry`: Parsed from MOD-01 UART telemetry string.
- `MotorDirection`: Integer enum aligned with MOD-01.
- `NavCommand`: Dataclass for motor instructions.

3.5.3 C# / Unity Layer

- `AcousticBeamStyle`: Enum controlling visual rendering mode.
- `AcousticBeamData`: Serialized payload for rendering.

3.6 Connections Between MOD-03 and Other Modules

- **MOD-01:** MOD-03 processes acoustics, and the STM32 appends `A_Ang` and `A_Hit` to the telemetry. `NavCommand` instructions are sent back via UART to control motors.
- **MOD-04:** MOD-04's FSM mirrors `fsm_acoustic_update()` and coordinates the transition to `ACOUSTIC_HOMING`.
- **MOD-05:** Telemetry is sent to Unity to render the acoustic beam via `MapManager_AcousticBeam`.

3.7 Functional Workflow

1. STM32 ADC samples 3 microphones at 8000Hz. 2. Filter applies IIR to remove noise. 3. Compute bearing via phase difference. 4. STM32 packs bearing into UART telemetry. 5. MOD-04 parses UART and creates AcousticTelemetry. 6. Homing bridge tracks consecutive hits. 7. FSM transitions when 3 hits confirm the bearing. 8. NavCommand is sent to STM32 to steer. 9. AugmentedStatusReport updates Unity visualization.

3.8 Risks and Design Considerations

- **Motor/gearbox noise:** Mitigated by the digital IIR filter.
- **Echo and reflection:** Mitigated by requiring 3 consecutive confirmations.
- **Rapid re-triggering:** Mitigated by a 3-second cooldown.
- **Phantom sounds:** Mitigated by a 30-second homing timeout.

4 MOD-04: Web Dashboard & Speech-to-Text Module

4.1 Module Task Distribution

Name	Student ID	Role	Responsibilities
Ömer Nacar	210104004814	Primary	Communication bridge design, telemetry JSON structure, WebSocket server
Tuana Melisa Aksoy	230104004903	Primary	Offline STT integration using Vosk / Whisper.cpp
Uğur Anıl Güney	210104004011	Secondary	Video streaming pipeline and dashboard communication
Fatma Öztürk	230104004152	Secondary	Operator command routing, system integration

Table 7: MOD-04 Team Responsibilities

4.2 Module Overview

MOD-04 is the communication and operator-interaction layer of the system. Its main responsibility is to connect the robot's internal software stack running on the Raspberry Pi 5 with the external operator interface. This module provides a Flask/WebSocket-based backend for telemetry and video streaming, routes operator-issued manual control commands into the robot control flow, and performs offline speech-to-text processing for voice-based control.

4.3 Internal Responsibilities

4.3.1 Web Dashboard Communication Subsystem

Defined through the `IWebDashboard` interface. It exposes four main methods:

- `start_server(...)`: Initializes the Flask and SocketIO server.
- `broadcast_telemetry(...)`: Sends current robot status to all clients.
- `stream_video_frame(...)`: Transmits compressed JPEG image frames.
- `on_operator_command_received(...)`: Processes commands coming from the operator.

4.3.2 Offline Speech-to-Text Subsystem

Defined through the `ISTTEngine` interface. It handles offline models:

- `load_offline_model(...)`: Loads the recognition model into Pi memory.
- `process_audio_blob(...)`: Converts raw audio bytes into `VoiceCommandData`.

4.4 Data Structures Used

4.4.1 AugmentedStatusReport

Telemetry structure transmitted from MOD-04 to the dashboard. Includes: `pos_x`, `pos_y`, `temperature`, `smoke_detected`, `victim_status`, `is_stuck`, `priority_level`, `acoustic_hit`, `acoustic_angle`.

4.4.2 VoiceCommandData

Output of the STT pipeline. Stores recognized `raw_text`, `intent`, and `confidence` score.

4.5 Connections Between MOD-04 and Other Modules

- **MOD-02:** Coordinates memory. MOD-04 calls `pause_vision_pipeline()` to free RAM before STT, then `resume_vision_pipeline()`.
- **MOD-05:** Telemetry, video, audio blobs, and commands flow between MOD-04 (backend) and MOD-05 (frontend).
- **MOD-03 & MOD-01:** Serves as a dissemination layer, packaging acoustic angles and embedded sensors into the `AugmentedStatusReport`.

4.6 Functional Workflow

1. Server starts. 2. Data is assembled into `AugmentedStatusReport`. 3. Broadcasts to Unity via WebSockets. 4. Streams JPEG video. 5. Routes manual overrides. 6. STT pipeline processes PTT audio. Vision pipeline is paused during speech inference to save RAM.

4.7 Risks and Design Considerations

Memory and performance pressure on the Raspberry Pi 5 during offline STT inference. Handled by pausing the vision pipeline. WebSocket payload constants must match the Unity client exactly.

5 MOD-05: Unity Digital Twin Module

5.1 Authors and Responsibilities

Name	Student ID	Role
Dicle Çoban	220104004088	Primary - UI, Map Visualization & Mocking
Nuri Ziya Kırtepe	210104004027	Secondary - WebSocket, JSON & Mocking
Evrim Doğa Solmaz	230104004042	Secondary - Audio Capture Pipeline

Table 8: MOD-05 Team Responsibilities

5.2 Executive Summary

Module 5 serves as the primary operator interface for the Rescue Robot system. Built using the Unity Engine, it provides a real-time "Digital Twin" visualization of the robot's state, environment, and mission progress. The module aggregates data from various sensors and presents them through a 2D/3D hybrid dashboard, allowing the operator to monitor telemetry and issue commands via Push-to-Talk (PTT).

5.3 System Architecture & Connections

Module 5 acts as the central data sink for the entire system, pulling telemetry from the backend (MOD-04) via Socket.IO, while streaming WAV audio blobs and commands back to the server. Internally, the architecture is event-driven through the `RobotManager`.

5.4 Script Deep Dive & Design Choices

- **RobotManager.cs:** Centralized coordinator. Distributes data to UI, Map, and Acoustic managers.
- **INetworkClient.cs:** Networking abstraction enabling swapping between Real and Mock data.
- **WebSocketClient.cs:** Handles Socket.IO using `ClientWebSocket`. Uses `SynchronizationConte` to marshal threads safely to Unity's Main Thread.
- **FileNetworkClient.cs:** Simulates live backend using local JSON files for demonstrations.
- **MapManager.cs:** Uses `GridToWorldPosition` and dictionary cell keys to prevent map pin clutter.
- **UIManager.cs:** HUD updates using a single `UpdateHUD` convenience method.
- **AudioManager.cs:** Implements manual WAV encoding (`EncodeToWav`) to provide 16-bit PCM required by the backend.
- **MapManager_AcousticBeam.cs:** Visualizes distress call direction with an acoustic beam.

- **VirtualSensor.cs & VictimInfo.cs:** Simulated environment objects that allow testing detection flows natively in Unity via `Physics.OverlapSphere`.

5.5 Data Contracts

The `TelemetryData` struct serves as the common language:

Field	Type	Description
posX / posY	float	2D Grid coordinates.
temperature	float	Environmental sensor data.
smokeDetected	bool	Fire safety indicator.
victimStatus	enum	AI classification result (Standing / Lying / Trapped).
acousticAngle	float	Direction of distress call.

Table 9: Core Telemetry Structure

5.6 Demo Mode & Presentation Readiness

The module features a dedicated Presentation Mode: 1. Toggle `Use Mock File Data` flag. 2. Disables live listeners. 3. Automatically loads `mock_telemetry.json`. 4. Plays back an automated rescue mission for office presentations.

5.7 Conclusion

Module 5 represents a robust, extensible visualization platform. Its decoupled architecture, interface-based networking, and integrated demonstration capabilities make it highly reliable for live operations and academic presentations.